Actividad 7. Algoritmos Voraces (Greedy)

Joaquin Carpio Arana

9 de noviembre de 2026

Repositorio de GitHub: https://github.com/EnderLove/Actividad7_Algoritmos_Voraces/

1 Introducción

Un algoritmo voraz (*greedy*) construye la solución paso a paso. En cada paso toma la mejor decisión local posible, sin volver atrás para cambiarla. Cuando el problema tiene la propiedad de elección voraz, es decir, cuando una buena decisión local puede extenderse a una solución óptima global, este tipo de algoritmo es correcto y, además, muy eficiente.

En este trabajo se resuelven dos problemas de optimización con esta estrategia, ambos implementados en C++: la mochila fraccional y la cobertura de antenas.

2 Problema 1: Mochila fraccional

2.1 Descripción

Se tiene una mochila con capacidad máxima W y n objetos. Cada objeto tiene un valor v_i y un peso w_i, y existe una sola vez. Se pueden tomar fracciones de un objeto. El objetivo es maximizar el valor total sin exceder W.

2.2 Estrategia greedy

La decisión local es elegir siempre el objeto con la **mayor razón valor/peso** (v_i/w_i) que todavía no se haya tomado, porque es el que aporta más valor por cada unidad de capacidad usada.

¿Por qué es correcta? Supongamos una solución óptima que deja sin usar parte de un objeto con mayor razón mientras usa parte de otro con menor razón. Si se cambia una pequeña cantidad de peso del objeto de menor razón por la misma cantidad del de mayor razón, el peso total no cambia y el valor total no disminuye (de hecho aumenta o queda igual). Por tanto, siempre existe una solución óptima que usa los objetos en orden decreciente de razón, que es justo lo que hace el algoritmo. Esta propiedad depende de poder tomar fracciones; si los objetos fueran indivisibles, el greedy ya no garantizaría el óptimo.

2.3 Algoritmo

1. Leer la capacidad W y los n objetos; calcular la razón v_i/w_i de cada uno.
2. Ordenar los objetos de mayor a menor razón (con `stable_sort`, de modo que los empates conserven el orden de entrada).
3. Sea $\mathit{restante} = W$ y $\mathit{valorTotal} = 0$. Recorrer los objetos en ese orden:
   - Si $\mathit{restante} = 0$, terminar.
   - Si $w_i \leq \mathit{restante}$: tomar el objeto completo, restar w_i a *restante* y sumar v_i al valor.

- Si no: tomar la fracción *restante*/w_i, sumar $v_i \cdot$ *restante*/w_i al valor y poner *restante* $= 0$.

4. Mostrar los objetos seleccionados, las fracciones usadas y el valor total.

El programa valida la entrada: la capacidad debe ser ≥ 0, debe haber al menos un objeto, los valores deben ser ≥ 0 y los pesos > 0.

2.4 Análisis de complejidad

Sea n el número de objetos.

- Leer los objetos y calcular las razones: $O(n)$.
- Ordenar por razón: $O(n \log n)$.
- Recorrido voraz: cada objeto se visita a lo más una vez y cada visita hace trabajo constante, así que cuesta $O(n)$.

El término dominante es el ordenamiento, por lo que la complejidad en tiempo es

$$O(n) + O(n \log n) + O(n) = O(n \log n).$$

En espacio se guarda el arreglo de objetos, así que la complejidad es $O(n)$.

2.5 Ejemplos de ejecución

Ejemplo 1 (capacidad 50; A: valor 60, peso 10; B: 100, 20; C: 120, 30). Las razones son 6, 5 y 4, así que el orden es A, B, C. A y B caben completos (peso 30) y quedan 20 unidades, que se usan en 20/30 de C: $60 + 100 + 80 = 240$.

```
$ ./mochila_fraccional examples/mochila1.txt
Capacidad maxima: 50
Objetos seleccionados:
A completo
B completo
20/30 Parte del objeto C
Valor total obtenido: 240
```

Ejemplo 2 (capacidad 25; A: 80, 20; B: 100, 10; C: 120, 30). Las razones son 4, 10 y 4. B va primero y cabe completo; quedan 15 unidades, que se usan en 15/20 de A: $100 + 60 = 160$.

```
$ ./mochila_fraccional examples/mochila2.txt
Capacidad maxima: 25
Objetos seleccionados:
B completo
15/20 Parte del objeto A
Valor total obtenido: 160
```

Caso extra: la capacidad alcanza para todo. Con capacidad 100 y objetos A (valor 10, peso 5) y B (valor 20, peso 10), se toman ambos completos.

```
Capacidad maxima: 100
Objetos seleccionados:
A completo
B completo
Valor total obtenido: 30
```

Caso extra: pesos decimales. Con capacidad 7.5, A (valor 10, peso 5) y B (valor 9, peso 4), B tiene mayor razón (2.25 contra 2). Se toma B completo y 3.5/5 de A: $9 + 7 = 16$.

```
Capacidad maxima: 7.5
Objetos seleccionados:
B completo
3.5/5 Parte del objeto A
Valor total obtenido: 16
```

Caso extra: capacidad 0. No se selecciona ningún objeto y el valor total es 0.

3 Problema 2: Cobertura de antenas

3.1 Descripción

Las casas están en posiciones enteras sobre una línea recta. Una antena colocada en la posición x con rango R cubre el intervalo $[x-R,\ x+R]$. Se busca colocar la **menor cantidad de antenas** de modo que todas las casas queden cubiertas.

3.2 Estrategia greedy

Con las casas ordenadas de izquierda a derecha, la decisión local es:

> Tomar la primera casa que aún no está cubierta, en la posición p, y colocar una antena en $p + R$.

Esa antena cubre $[p,\ p + 2R]$. Se ubica lo más a la derecha posible sin dejar fuera a la casa p, de modo que cubre la mayor cantidad de casas que aún faltan.

¿Por qué es correcta? Todas las casas a la izquierda de p ya están cubiertas, así que cualquier solución debe tener una antena que cubra a p, y esa antena está en una posición $x \leq p + R$. Si se mueve esa antena a $p + R$, sigue cubriendo a p y su intervalo se desplaza hacia la derecha, de modo que cubre al menos las mismas casas de las que faltan (las que están a la derecha de p). Por tanto, se puede reemplazar sin aumentar la cantidad de antenas. Repitiendo el argumento con la siguiente casa sin cubrir, la solución voraz usa a lo más tantas antenas como una solución óptima.

3.3 Algoritmo

1. Leer R y las posiciones de las casas.
2. Ordenar las posiciones de menor a mayor.
3. Sea $i = 0$. Mientras $i < n$:
   - Colocar una antena en $posAntena = casas[i] + R$.
   - Avanzar i mientras $casas[i] \leq posAntena + R$, es decir, mientras la casa esté dentro del alcance de la antena recién colocada.
4. Mostrar las posiciones de las antenas y la cantidad total.

3.4 Análisis de complejidad

Sea n el número de casas.

- Ordenar las posiciones: $O(n \log n)$.
- Recorrido voraz: aunque hay dos ciclos anidados, el índice i solo avanza y nunca retrocede. Cada casa se examina una sola vez, así que el recorrido completo cuesta $O(n)$.

La complejidad en tiempo es

$$O(n \log n) + O(n) = O(n \log n).$$

En espacio se guardan las posiciones de las casas y las de las antenas (a lo más n), es decir, $O(n)$.

3.5 Ejemplos de ejecución

Ejemplo 1 (casas $[1, 2, 7, 11, 20, 21, 30]$, $R = 5$). La casa 1 queda sin cubrir, se coloca una antena en 6, que cubre $[1, 11]$, es decir, las casas 1, 2, 7 y 11. La siguiente casa sin cubrir es la 20, se coloca una antena en 25, que cubre $[20, 30]$ y alcanza a las casas 20, 21 y 30.

```
$ ./cobertura_antenas examples/antenas1.txt
Antenas colocadas en:
6
25
Cantidad total: 2
```

Ejemplo 2 (casas $[2, 4, 8, 15, 18, 22]$, $R = 3$). La casa 2 lleva una antena en 5, que cubre $[2, 8]$ (casas 2, 4 y 8). La casa 15 lleva una antena en 18, que cubre $[15, 21]$ (casas 15 y 18). La casa 22 lleva una antena en 25.

```
$ ./cobertura_antenas examples/antenas2.txt
Antenas colocadas en:
5
18
25
Cantidad total: 3
```

Caso extra: entrada desordenada y con casas repetidas. Con casas $[30, 1, 1, 10, 12, 3]$ y $R = 2$, el programa primero las ordena. Las antenas quedan en 3 (cubre 1 a 5, incluyendo las dos casas en 1 y la casa 3), en 12 (cubre 10 a 14) y en 32 (cubre 30 a 34).

```
Antenas colocadas en:
3
12
32
Cantidad total: 3
```

Caso extra: una sola casa. Con casa $[100]$ y $R = 4$ se coloca una antena en 104.

Caso extra: $R = 0$. Cada posición distinta necesita su propia antena. Con casas $[5, 5, 7]$ se obtienen las antenas 5 y 7 (cantidad total: 2).

Validación. Además de los ejemplos, el programa de antenas se comparó contra una búsqueda exhaustiva (fuerza bruta) en 150 casos aleatorios pequeños, y la cantidad mínima de antenas coincidió en todos.

4 Resumen de complejidades

Problema	**Tiempo**	**Espacio**	**Paso dominante**
Mochila fraccional	$O(n \log n)$	$O(n)$	Ordenar por valor/peso
Cobertura de antenas	$O(n \log n)$	$O(n)$	Ordenar las posiciones

En ambos problemas, el recorrido voraz es lineal y el costo total lo determina el ordenamiento inicial.

5 Conclusiones

La estrategia greedy resolvió ambos problemas de forma óptima y eficiente. En la mochila fraccional, la clave es ordenar por valor/peso y poder tomar fracciones. En la cobertura de antenas, la clave es colocar cada antena lo más a la derecha posible respecto de la primera casa sin cubrir. En los dos casos, ordenar los datos primero convierte el problema en un recorrido de una sola pasada.